TP1

- Explicar qué es un socket y los diferentes tipos de sockets.

En sistemas Linux/Unix, los programas realizan operaciones de entrada/salida (E/S) mediante el uso de **descriptores de archivo**. Un descriptor de archivo es un número entero que el sistema operativo asigna a un recurso abierto, como puede ser un archivo en disco, una terminal, una tubería (*pipe*), una cola FIFO o incluso una conexión de red.

En este contexto, un **socket** es un tipo especial de descriptor de archivo que permite establecer una comunicación entre procesos, ya sea en la misma máquina o a través de una red.

Para trabajar con sockets, se utiliza la llamada al sistema `socket()`, que crea un nuevo socket y devuelve un descriptor de archivo asociado a la comunicación. A partir de este descriptor, el programa puede utilizar distintas llamadas al sistema para enviar y recibir datos, tales como:

- `send()` y `recv()` (específicas para sockets)
- `read()` y `write()` (genéricas de entrada/salida)

De esta forma, los sockets integran la comunicación de red dentro del mismo modelo de E/S que utiliza el sistema operativo, permitiendo tratar una conexión de red de manera similar a un archivo.

Tipos de sockets:

- Raw sockets(sockets puros) “SOCK_RAW”
- Stream sockets (sockets de flujo) “SOCK STREAM”
- Datagram sockets (sockets de datagramas) “SOCK DGRAM”

- Cuáles son las estructuras necesarias para operar con sockets en el modelo C-S y cómo se hace para ingresar los datos requeridos. -- Explicar con un ejemplo.

En el modelo Cliente-Servidor (C-S), la comunicación mediante sockets requiere el uso de ciertas estructuras que permiten definir la dirección y el puerto de conexión. La más importante es la estructura `sockaddr_in`, utilizada para direcciones IPv4.

```
struct sockaddr_in {

    short sin_family;        // Tipo de dirección (AF_INET)

    unsigned short sin_port; // Puerto

    struct in_addr sin_addr; // Dirección IP

};
```

Un aspecto fundamental al trabajar con sockets es la **ordenación de bytes**.

Existen dos formas de almacenar números en memoria:

- **Big Endian** → primero el byte más significativo
- **Little Endian** → primero el byte menos significativo

La red utiliza un estándar llamado:

Network Byte Order (Big Endian)

Sin embargo, no todas las máquinas utilizan ese formato internamente (Host Byte Order). Por eso, cuando se envían datos por red, es necesario convertirlos.

Ejemplo de carga de datos

struct sockaddr_in servidor;

servidor.sin_family = AF_INET;

servidor.sin_port = htons(8080); // Conversión a Network Byte Order

servidor.sin_addr.s_addr = INADDR_ANY;

- Explicar modo bloqueante y no bloqueante en sockets y cuáles son los “sockets calls” a los cuales se pueden aplicar estos modos. -- Explicar las funciones necesarias.

Modo bloqueante

Por defecto, los sockets se crean en modo bloqueante. Esto significa que ciertas funciones **detienen la ejecución del programa** hasta que ocurre un evento.

Por ejemplo:

- `accept()` → espera hasta que un cliente se conecte
- `recv()` o `recvfrom()` → espera hasta que lleguen datos

Si no hay datos disponibles, el programa queda “esperando” en esa instrucción.

Modo no bloqueante

En modo no bloqueante, las funciones **no esperan**.
Si no hay datos disponibles:

- la función devuelve `-1`
- y se establece un error (`errno = EWOULDBLOCK`)

Es decir, el programa sigue ejecutándose sin quedarse detenido.

Socket calls que pueden ser bloqueantes

Las principales funciones que pueden operar en modo bloqueante son:

- `accept()`

- `recv()`
- `recvfrom()`
- `send()`
- `connect()`

Estas funciones están diseñadas para bloquearse y el sistema operativo las configura así por defecto al crear el socket.

- Describir el modelo C-S aplicado a un servidor con Concurrencia Real.

El modelo Cliente-Servidor (C-S) consiste en un sistema donde un **servidor** espera solicitudes de múltiples **clientes** y responde a cada uno de ellos.

En su forma más simple, el servidor atiende a un cliente por vez. Sin embargo, esto no es eficiente cuando hay múltiples clientes, ya que los demás deben esperar. Para solucionar esto se utiliza la **concurrencia real**.

La concurrencia real permite que el servidor **atienda múltiples clientes al mismo tiempo**, en lugar de hacerlo de manera secuencial. Cada cliente es manejado de forma independiente.

1. El servidor crea un socket con `socket()`
2. Asocia una dirección con `bind()`
3. Escucha conexiones con `listen()`
4. Acepta clientes con `accept()`
5. Por cada cliente, crea un nuevo proceso o hilo

Formas de implementar la concurrencia

- Mediante procesos (fork()) → el servidor crea un proceso hijo para cada cliente
- Mediante hilos (threads) → se crea un hilo por cada cliente
- Multiplexación (select) ⟶ En lugar de crear procesos o hilos, el servidor usa `select()` para manejar múltiples conexiones en un solo proceso.

- Escribir un ejemplo en lenguaje C distinto del ya presentado.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PUERTO 8080

int main() {
    int sockfd, cliente;
    struct sockaddr_in servidor;
    char buffer[1024] = {0};

    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    servidor.sin_family = AF_INET;
    servidor.sin_port = htons(PUERTO);
    servidor.sin_addr.s_addr = INADDR_ANY;

    bind(sockfd, (struct sockaddr*)&servidor, sizeof(servidor));
    listen(sockfd, 3);

    printf("Servidor esperando conexiones...\n");

    cliente = accept(sockfd, NULL, NULL);

    read(cliente, buffer, sizeof(buffer));
    printf("Mensaje recibido: %s\n", buffer);

    char *respuesta = "Mensaje recibido correctamente";
    write(cliente, respuesta, strlen(respuesta));

    close(cliente);
    close(sockfd);

    return 0;
}
```

```
miabertola@Mia:~/PDI_2025-TP1-TP2-TP3-TP4/TP1/Có
digos en C$ ls
miabertola@Mia:~/PDI_2025-TP1-TP2-TP3-TP4/TP1/Có
digos en C$ gcc servidor_base.c -o servidor_base

gcc cliente_base.c -o cliente_base
miabertola@Mia:~/PDI_2025-TP1-TP2-TP3-TP4/TP1/Có
digos en C$ ./servidor_base
Socket creado...
Se hace el socket bind ..
Servidor en modo escucha ...
El servidor acepta al cliente ...
Del cliente: Hola
        : Como estas
Del cliente: Bien vos
        :
```

```
miabertola@Mia:~/PDI_2025-TP1-TP2-TP3-TP4/TP1/Có
digos en C$ gcc servidor_simple.c -o servidor_si
mple
gcc cliente_simple.c -o cliente_simple
miabertola@Mia:~/PDI_2025-TP1-TP2-TP3-TP4/TP1/Có
digos en C$ gcc servidor_base.c -o servidor_base

gcc cliente_base.c -o cliente_base
miabertola@Mia:~/PDI_2025-TP1-TP2-TP3-TP4/TP1/Có
digos en C$ ./cliente_base
Socket creado ..
Conectado al servidor..
Ingrese texto : Hola
Servidor : Como estas
Ingrese texto : Bien vos
```

- Cómo se cierra una conexión C-S ?. Métodos que eviten la pérdida de información.

En una comunicación Cliente-Servidor (C-S), el cierre de la conexión es una etapa importante para asegurar que **todos los datos hayan sido enviados y recibidos correctamente**.

La forma más simple de cerrar una conexión es utilizando la función close(sockfd) → ésta llamada libera el descriptor de archivo asociado al socket y finaliza la conexión.

Si se utiliza `close()` directamente puede haber datos en tránsito y podrían perderse

Para evitar pérdida de información, se utiliza la función shutdown(sockfd, SHUT_WR) → esto indica que no se enviarán más datos pero se siguen recibiendo datos pendientes.

- Probar el código del chat entre cliente y servidor. Cambiar tipo de socket y volver a probar.

home > miabertola > PDI_2025-TP1-TP2-TP3-TP4 > TP1 > Códigos en C > C servidor_udp.c > main()

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PUERTO 8080

int main() {
    int sockfd;
    char buffer[1024];
    struct sockaddr_in servidor, cliente;
    socklen_t len = sizeof(cliente);

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    servidor.sin_family = AF_INET;
    servidor.sin_port = htons(PUERTO);
    servidor.sin_addr.s_addr = INADDR_ANY;

    bind(sockfd, (struct sockaddr*)&servidor, sizeof(servidor));

    printf("Servidor UDP esperando mensajes...\n");

    recvfrom(sockfd, buffer, sizeof(buffer), 0,
             (struct sockaddr*)&cliente, &len);

    printf("Mensaje recibido: %s\n", buffer);

    char *respuesta = "Mensaje recibido (UDP)";
    sendto(sockfd, respuesta, strlen(respuesta), 0,
           (struct sockaddr*)&cliente, len);

    close(sockfd);
    return 0;
}
```

home > miabertola > PDI_2025-TP1-TP2-TP3-TP4 > TP1 > Códigos en C > C cliente_udp.c > main()

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PUERTO 8080

int main() {
    int sockfd;
    char buffer[1024];
    struct sockaddr_in servidor;
    socklen_t len = sizeof(servidor);

    sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    servidor.sin_family = AF_INET;
    servidor.sin_port = htons(PUERTO);
    inet_pton(AF_INET, "127.0.0.1", &servidor.sin_addr);

    char *mensaje = "Hola desde cliente UDP";

    sendto(sockfd, mensaje, strlen(mensaje), 0,
           (struct sockaddr*)&servidor, len);

    recvfrom(sockfd, buffer, sizeof(buffer), 0,
             NULL, NULL);

    printf("Respuesta del servidor: %s\n", buffer);

    close(sockfd);
    return 0;
}
```

```
miabertola@Mia:~/PDI_2025-TP1-TP2-TP3-TP4/TP1/Có
digos en C$ gcc servidor_udp.c -o servidor_udp
gcc cliente_udp.c -o cliente_udp
miabertola@Mia:~/PDI_2025-TP1-TP2-TP3-TP4/TP1/Có
digos en C$ ./servidor_udp
Servidor UDP esperando mensajes...
Mensaje recibido: Hola desde cliente UDP
miabertola@Mia:~/PDI_2025-TP1-TP2-TP3-TP4/TP1/Có
digos en C$
```

```
Ingrese texto : ^C
miabertola@Mia:~/PDI_2025-TP1-TP2-TP3-TP4/TP1/Có
digos en C$ gcc servidor_udp.c -o servidor_udp
gcc cliente_udp.c -o cliente_udp
miabertola@Mia:~/PDI_2025-TP1-TP2-TP3-TP4/TP1/Có
digos en C$ ./cliente_udp
Respuesta del servidor: Mensaje recibido (UDP)
miabertola@Mia:~/PDI_2025-TP1-TP2-TP3-TP4/TP1/Có
digos en C$
```